

Convolutional Neural Networks

A Complete Visual Guide

From Pixels to Predictions — covering convolution, pooling, backpropagation,
and beyond

1 **What is a CNN?**
Motivation & architecture overview

2 **Core Operations**
Convolution, filters, feature maps

3 **Non-linearity & Pooling**
ReLU activation and downsampling

4 **Full Architecture**
End-to-end pipeline walkthrough

5 **Training CNNs**
Loss, backprop, gradient descent

6 **Regularisation**
Dropout and data augmentation

7 **What CNNs Learn**
Hierarchical feature visualisation

#1 What is a CNN?

A **Convolutional Neural Network (CNN)** is a deep learning architecture designed specifically for grid-like data — primarily images. Rather than treating each pixel independently, CNNs learn spatial hierarchies of features through weight-sharing and local connectivity.

Key insight: CNNs learn features automatically — you don't hand-craft edge detectors or shape templates. The network discovers them during training.

Hierarchical Feature Learning

CNNs build understanding progressively, from low-level to high-level:

Stage	What is detected	Examples
Early layers	Low-level patterns	Edges, corners, lines
Middle layers	Mid-level structures	Textures, shapes, curves
Deep layers	High-level parts	Eyes, wheels, handles
Final layer	Semantic concepts	Cat, Dog, Car, Person

Why Not a Standard Neural Network?

A standard 224×224×3 image has 150,528 pixel values. Connecting these to even 100 neurons requires 15 million weights — just for the first layer! CNNs solve this with:

- Weight sharing — same filter applied across the whole image
- Local receptive fields — each neuron sees only a small patch
- Pooling — progressively reduce spatial dimensions

Network type	Input size	1st-layer neurons	Parameters
Fully connected	224×224×3	100	15,052,800
CNN (3×3 filter)	224×224×3	32 filters	864

#2

Core Operations: Convolution

Convolution is the fundamental operation of a CNN. A small matrix called a **kernel** (or filter) slides over the image, computing dot products at each position to produce a **feature map**.

Convolution Diagram



The blue highlighted patch shows the first 3×3 region of the input being element-wise multiplied with the kernel. The results are summed to produce one value in the output feature map. The kernel then slides by the **stride** to compute the next value.

Stride

Stride controls how many pixels the kernel moves each step:

- Stride = 1 → move one pixel at a time (more overlap, larger output)
- Stride = 2 → skip every other pixel (smaller output, faster)

Output size formula:

```
out = floor((in - kernel + 2×pad) / stride) + 1
```

Padding

Every convolution without padding shrinks the image. Adding zeros around the border preserves dimensions:

- Valid padding → no padding, output shrinks
- Same padding → output = input size

Zero-padded 3×3 becomes 5×5:

```
[ 0 0 0 0 0 ] [ 0 1 2 3 0 ] [ 0 4 5 6 0 ] [ 0 7 8 9 0 ] [ 0 0 0 0 0 ]
```

Kernel Examples

Different kernels detect different patterns:

Vertical Edge

1	0	-1
1	0	-1
1	0	-1

Horizontal Edge

1	1	1
0	0	0
-1	-1	-1

Sharpen

0	-1	0
-1	5	-1
0	-1	0

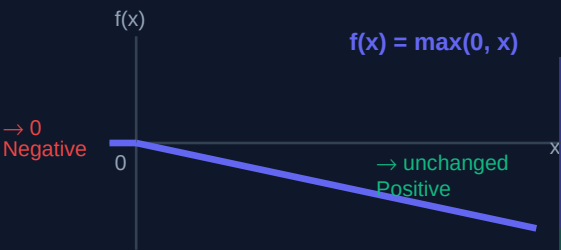
Blur / Average

1	1	1
1	1	1
1	1	1

#3 Non-Linearity & Pooling

ReLU Activation

After convolution, the **Rectified Linear Unit (ReLU)** introduces non-linearity by zeroing out negatives



- Negative inputs → 0
- Positive inputs → unchanged
- Prevents vanishing gradients
- Computationally very cheap

Max Pooling

Max Pooling reduces spatial dimensions by summarising regions — max pooling takes the maximum value from each patch:



- Reduces computation in deeper layers
- Adds translation invariance
- Retains the strongest activations
- 2x2 pool with stride 2 halves each dimension

ReLU Example

Input values	After ReLU
-3.2	0.0
5.1	5.1
-0.8	0.0
8.4	8.4
0.0	0.0

#4 The Full CNN Architecture

The Complete Pipeline



Each stage transforms the data: convolutional layers extract features, ReLU introduces non-linearity, pooling compresses, and fully-connected layers perform classification.

Stage	Input → Output	Purpose
Conv Layer (32 filters)	224×224×3 → 224×224×32	Extract 32 types of local patterns
ReLU	224×224×32 → 224×224×32	Zero out negative activations
Max Pool 2×2	224×224×32 → 112×112×32	Halve spatial dims, keep max values
Conv Layer (64 filters)	112×112×32 → 112×112×64	Learn more complex features
ReLU	112×112×64 → 112×112×64	Non-linearity again
Max Pool 2×2	112×112×64 → 56×56×64	Halve spatial dims again
Flatten	56×56×64 → 200,704	Convert 3D tensor to 1D vector
Dense (256 units)	200,704 → 256	High-level reasoning
Softmax (N classes)	256 → N	Probabilities over classes

Softmax: Turning Scores into Probabilities

The final layer outputs raw scores (logits). Softmax normalises these into a proper probability distribution that sums to 1.0:

Class	Raw Score	After Softmax
Dog	3.0	0.84
Cat	1.0	0.11
Horse	0.0	0.05
Total	—	1.00 ✓

#5

Training CNNs

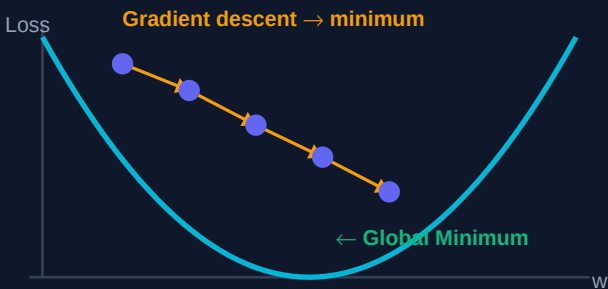
Loss Function — Cross-Entropy

The loss function quantifies how wrong the network's prediction is. For multi-class classification, **cross-entropy loss** is used:

Scenario	True class	Prediction	Loss
Bad prediction	Cat	Cat: 0.2 / Dog: 0.8	High ↑
Good prediction	Cat	Cat: 0.95 / Dog: 0.05	Low ↓

Gradient Descent — Rolling Downhill

Gradient descent updates weights to minimise loss. Imagine the loss surface as a landscape — we want to roll to the lowest point:



Each weight is updated as: $w_{\text{new}} = w_{\text{old}} - \eta \times (\partial L / \partial w)$, where η (eta) is the learning rate.

Learning Rate Guide

Learning Rate	Behaviour	Risk
0.0001 (very small)	Slow, stable convergence	May get stuck, very slow
0.001 (typical)	Balanced speed and stability	Good default starting point
0.01	Fast early, may oscillate	Can overshoot valleys
1.0 (too large)	Jumps wildly	Diverges, loss explodes

Epochs & Batches

Epoch

One complete pass through the entire training dataset. Networks typically train for 10–200 epochs. image, batches process multiple images before updating.

- 10,000 images × 50 epochs = 500,000 forward passes
- Loss should decrease each epoch

Mini-Batch

- Batch size 32: update every 32 images
- Smoother gradients, more stable training
- Typical values: 16, 32, 64, 128

#6

Preventing Overfitting

What is Overfitting?

Overfitting happens when the model memorises training examples rather than learning general patterns. A classic sign:

Metric	Training Set	Test Set	Verdict
Overfit model	99%	70%	■ Memorised, not learnt
Good model	92%	90%	✓ Generalises well

Dropout

During training, dropout **randomly deactivates** neurons with probability p . This forces the network to not rely on any single neuron, building robust representations.

- dropout=0.5 → randomly zero out 50% of neurons
- Applied only during training, not at test time
- Equivalent to training an ensemble of sub-networks
- Common values: 0.3 – 0.5

Data Augmentation

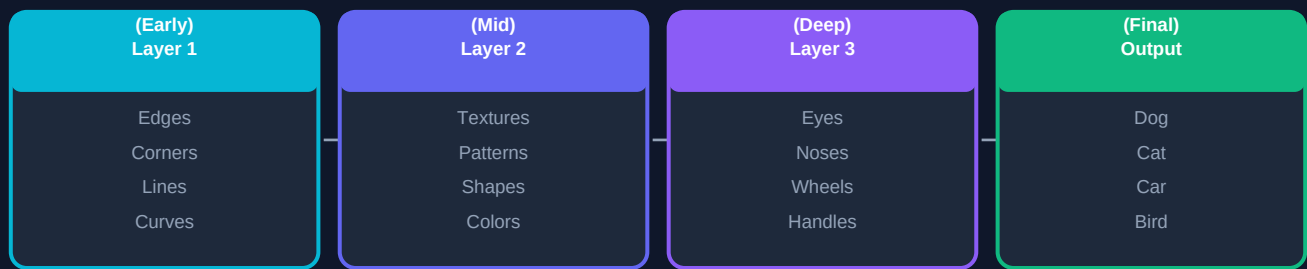
Artificially expand the training set by applying **random transformations** to existing images — the label stays the same but the pixel data changes.

- Horizontal/vertical flip
- Random rotation (e.g. $\pm 15^\circ$)
- Random crop and resize
- Colour jitter (brightness, contrast, saturation)
- Gaussian noise / blur

Rule of thumb: Always use data augmentation and dropout together for best regularisation.

#7 What CNNs Learn

CNNs build a hierarchy of increasingly abstract representations. Visualising intermediate activations reveals what each layer has learnt to detect:



Famous CNN Architectures

The field has evolved rapidly. Here are the landmark architectures every practitioner should know:

Architecture	Year	Key Innovation	Top-5 Error
LeNet-5	1998	Pioneered conv + pool + dense pipeline	N/A
AlexNet	2012	Deep CNN on GPU, ReLU, dropout	15.3%
VGG-16	2014	Very deep (16 layers), 3×3 convs only	7.3%
ResNet-50	2015	Residual (skip) connections, 50 layers	5.3%
EfficientNet	2019	Compound scaling of width/depth/res	2.9%

Recommended learning path: Master the basic pipeline first (Sections 1–5), then implement a simple CNN in PyTorch or TensorFlow before studying AlexNet → ResNet.